

Inverted Indexing, Sparse Retrieval, and the Vocabulary Mismatch Problem

Tentative Deadline: 5th September, 2026

In this assignment, you have to perform the following tasks:

- Index and Retrieve with Pyserini
- Inspect data for vocabulary mismatch
- Explore Pseudo-Relevance Feedback & LLM-generated Feedback
- Sparse Retrieval with the Splade model.

Datasets

All datasets are drawn from the BEIR benchmark. Use Pyserini's / ir_datasets' standard loaders rather than re-scraping or reformatting the raw sources yourself.

- **SciFact** — small corpus of ~5K documents.
- **FEVER, HotpotQA** — ~5 million documents.
- **MSMARCO (passage style, 8.8 million corpus items)** — the largest corpus of all the datasets here, **only for extra credit**.

***Report deliverable:** for each of the main datasets, one line giving corpus size, number of queries, and any deviation from the standard split you actually used. This will help with input in later tables.*

Part 1: Build an Index with Pyserini

Use Pyserini to build a standard Lucene inverted index over each corpus.

- Build one index per dataset (SciFact, FEVER, HotpotQA).
- Use Pyserini's IndexReader API to pull the term-frequency vector, document length, and corpus-level document frequency. These statistics are required for Part 4a's pseudo-relevance feedback implementation.

***Report deliverable:** index build time and on-disk index size for each of the three datasets.*

Part 2: Sparse Retrieval Baselines

Use Pyserini's LuceneSearcher to run retrieval over each Part 1 index.

- Report results with Pyserini's default BM25 parameters, then re-tune k1/b and justify your chosen values.
- Switch to classic TF-IDF similarity and compare.

Evaluate on all three required datasets (SciFact, FEVER, HotpotQA).

Report deliverable: one table per dataset (or one combined table with a dataset column) reporting $nDCG@10$, $Recall@100$, $MRR@10$, and MAP for (a) default-parameter BM25, (b) tuned BM25, and (c) TF-IDF. State the tuned k_1/b values, how you searched for them (grid, per-dataset or shared), and 2-3 sentences on what the tuning changed and why.

Part 3: Seeing Vocabulary Mismatch

- Pull cases where BM25 fails to retrieve the gold document in the top-k. Characterize why they fail (examples could be synonymy, paraphrasing, abbreviation vs. expansion, etc.).
- Compute lexical overlap (Jaccard coefficient over query and gold-document tokens) and compare the distribution for query–document pairs where BM25 succeeds vs. fails. Report whether low overlap predicts failure, and how strongly.
- Nice to read (but not necessary): this [paper by CMU LTI](#) on the original description of the problem.

Report deliverable: per dataset, a breakdown of failure cases of different types and 2-3 concrete query/gold-document examples for each type you find; a plot or summary statistics comparing the Jaccard-overlap distribution for BM25-success vs. BM25-failure pairs; and a short written verdict on whether, and how strongly, low overlap predicts failure.

Part 4: Pseudo-Relevance Feedback — Corpus-Retrieved and LLM-Generated

Build 4a first, then reuse it unchanged in 4b.

4a. Rocchio & RM3 (corpus-retrieved feedback)

Implement Rocchio's algorithm yourself, using Part 1's IndexReader to pull term-frequency vectors for feedback documents:

$$w_t = \alpha \cdot f(q)[t] + (\beta / N) \cdot \sum_{d \in \text{feedback docs}} \tilde{f}(d)[t]$$

- Run Part 2's BM25 retrieval, take the top-N results as pseudo-relevant, and filter out overly common corpus terms (e.g. terms appearing in more than 10% of documents) before ranking candidate expansion terms.
- Construct the re-weighted query as a boosted query via Pyserini's Query Builder API (rather than string concatenation) and re-search.
- You may also use Pyserini's built-in `set_rm3()` as a second feedback model and as a sanity check on your own Rocchio numbers.
- Report results across at least two settings of N (feedback documents) and k (expansion terms kept), and discuss query drift — cases where feedback pulls the query toward an unrelated term.

Report deliverable: a metrics table ($nDCG@10$, $Recall@100$, $MRR@10$, MAP) for Rocchio and for RM3 across your ≥ 2 (N, k) settings, per dataset, compared against the Part 2 BM25 baseline. Include at least two concrete query-drift examples — the added term(s) and why they hurt that query.

4b. HyDE (LLM-generated feedback)

HyDE ([Gao et al., 2023](#)) prompts an LLM to generate a hypothetical answer document for the query, then treats that document's terms as expansion terms: the LLM's generated text stands in for the corpus-retrieved feedback documents from 4a.

- Use a small open-weight LLM you can run locally (e.g. a 7B-class (Qwen) model via vLLM, no need to use a paid API) to sample a few hypothetical documents per query.
- Reuse your 4a Rocchio/RM3 implementation unchanged — only the feedback source (LLM instead of corpus retrieval) changes.
- Compare three variants: naive concatenation of query + hypothetical documents, your Rocchio/RM3-weighted version, and 4a's corpus-based PRF. Report which matters more: the feedback source (LLM-generated vs. corpus-retrieved) or the combination method (concatenation vs. feedback model).

Report deliverable: a metrics table (same four metrics) for the three variants — naive concatenation, Rocchio/RM3-weighted HyDE, and 4a's corpus PRF — per dataset, plus a short written answer, grounded in the table rather than asserted, to whether feedback source or combination method matters more.

Part 5: Sparse Retrieval with a Pretrained SPLADE Checkpoint

Take a look at the [SPLADE paper](#) and familiarize yourself. Load a pretrained checkpoint from HuggingFace — `naver/splade-v3`, or an earlier model such as `naver/splade-cocondenser-ensembledistil` — and use it for inference.

For a BERT-like encoder with an MLM head, the sparse representation assigns each vocabulary term j a weight:

$$w_j = \max_i \log(1 + \text{ReLU}(o_{ij}))$$

where o_{ij} is the MLM output logit for vocabulary term j at position i . Pyserini's `SpladeQueryEncoder` computes this on the query side; check whether a prebuilt SPLADE impact index already exists for your dataset (for Pyserini) before building your own with `--impact --pretokenized`.

- Encode and index all three required corpora (SciFact, FEVER, HotpotQA) with your chosen checkpoint.
- Run retrieval and report the same metrics as every other part.
- Compare directly against Part 4: for at least 10 queries per dataset, list the top expansion terms SPLADE assigns non-zero weight to beyond the literal query terms, alongside the Rocchio/RM3 terms from 4a and the HyDE terms from 4b for the same queries. Report the term overlap across the three sources and discuss 2-3 cases where they disagree.

Report deliverable: a metrics table for SPLADE per dataset, plus the ≥ 10 -query, three-way expansion-term comparison table (SPLADE vs. Rocchio/RM3 vs. HyDE) with the overlap stated and the disagreement cases discussed.

Evaluation Procedure

Report the same metrics for Part 2, Part 4 (4a and 4b), and Part 5, per dataset (SciFact, FEVER, HotpotQA):

- **nDCG@10** (primary metric, standard for BEIR)
- **Recall@100**
- **MRR@10**
- **MAP**

Extra Credit (3 points total)

1. Document-Side Expansion (1 point). Everything so far expands the query at retrieval time. An alternative is to expand the document at indexing time: predict pseudo-queries a document is likely to be retrieved by, and append them before indexing (Nogueira et al.'s [doc2query](#) / [docTTTTTquery](#)). Prompt the same LLM from Part 4b to generate a handful of plausible queries per document, append them to the document text, rebuild the index, and re-run Part 2's BM25 baseline over it. Report whether this recovers Part 3 failure cases that query-side expansion (Part 4) didn't, and at what cost — index size and build time — versus query-side expansion's recurring per-query cost.

Report deliverable: BM25 metrics before vs. after doc2query expansion, the index-size, and a count of how many Part 3 failure cases the expanded index is able to recover, if at all.

2. Full-Scale MSMARCO (1 point). Repeat Parts 1, 2, and 5 (index, BM25 baseline, SPLADE) on the full MSMARCO passage corpus. Pyserini ships a prebuilt SPLADE++ impact index for MSMARCO, so Part 5 here is largely running the existing pipeline at scale rather than rebuilding it. Report timing and index size alongside retrieval metrics; this item is scored on whether your Part 1/2/5 pipeline runs unmodified at MSMARCO's scale, not on beating the SciFact/FEVER/HotpotQA numbers.

Report deliverable: index build time, retrieval latency (per query), index size, and the Part 2/5 metrics table, all on MSMARCO.

3. Training SPLADE (1 point). Instead of the pretrained checkpoint from Part 5, fine-tune your own SPLADE model: a contrastive ranking loss with in-batch negatives, plus a FLOPS sparsity regularizer applied separately to the average query and document vectors, with the regularization weight scheduled (e.g. quadratic warm-up) rather than fixed from the start. Compare your trained model's expansions against the pretrained checkpoints from Part 5 — where do they agree, and where does training on your specific corpora change what gets expanded to?

Report deliverable: a metrics table for your trained model vs. the Part 5 pretrained checkpoint, per dataset, plus a short comparison of where the expansion terms agree and where training on your corpora changed them.

General Hints

- Check [Pyserini's prebuilt-index catalog](#) — SciFact, FEVER, and HotpotQA may already have prebuilt BM25 and SPLADE++ impact indexes available, which is useful as a sanity check on your own indexing.
- Use Kaggle over Colab where possible for GPU quota; terminate GPU sessions when idle.
- For Part 4b (and extra-credit document expansion), a 7B-class open-weight model served locally (VLLM) is enough — this is the model size the HyDE feedback paper found effective, and it keeps you off paid APIs entirely.

Submission Protocol

Moodle OR Google Forms (Drive upload) will be used for submission.

- All findings go into a single report file.
- Zip all code (indexing, retrieval, feedback, SPLADE inference — and training, if attempting extra-credit) together with the report into one file, named [Roll1_Roll2].(zip/tar.gz).
- Only one team member submits.

Summary of Report Deliverables

One report file, structured to mirror the assignment parts. Checklist:

- Part 1 — index build time and size, per dataset.
- Part 2 — nDCG@10 / Recall@100 / MRR@10 / MAP for default BM25, tuned BM25, and TF-IDF, per dataset; tuned k1/b values and justification.
- Part 3 — failure-case breakdown by category with examples; Jaccard-overlap comparison (success vs. failure); written verdict on overlap-failure correlation.
- Part 4a — Rocchio/RM3 metrics at ≥ 2 (N,k) settings, per dataset, vs. the Part 2 baseline; ≥ 2 query-drift examples.
- Part 4b — metrics for naive concatenation / Rocchio-RM3-weighted HyDE / 4a's corpus PRF, per dataset; written verdict on feedback source vs. combination method.
- Part 5 — SPLADE metrics per dataset; ≥ 10 -query expansion-term comparison across SPLADE, Rocchio/RM3, and HyDE.
- Extra credit 1 (if attempted) — BM25 metrics before/after doc2query expansion; index size; failure cases recovered.
- Extra credit 2 (if attempted) — MSMARCO timing, index size, and Part 2/5 metrics.
- Extra credit 3 (if attempted) — trained-vs-pretrained SPLADE metrics; expansion-term agreement discussion.

Code (indexing, retrieval, feedback, SPLADE inference, and training if applicable) is zipped separately alongside the report per the Submission Protocol above — it is not part of the report file itself.

LLM Usage Policy

Any use of LLM assistants such as Claude, ChatGPT, Gemini, etc. needs to be **documented in full length in your reports**. You will also have to provide **shareable links to all chats** which you used to implement this programming assignment. Using these assistants and not providing this information goes against the course's honor code.